



INSTITUTO POLITÉCNICO NACIONAL
ESCUELA SUPERIOR DE CÓMPUTO



Sesión de Laboratorio 7:

FIRMA CON RSA-CRT DE LIBRO DE TEXTO



SELECTED TOPICS OF CRYPTOGRAPHY

Profesora || **Sandra Díaz Santiago**

Grupo || **7CM1**

Alumno || **Areli Alejandra Guevara Badillo**

21 de abril de 2026

[illegible]

2. Función de Firma

```
# Funcion de firma RSA-CRT

def firmar_archivo(nombre_archivo, priv_key_b64):
    priv_data = json.loads(base64.b64decode(priv_key_b64))

    with open(nombre_archivo, "rb") as f:
        h_obj = SHA256.new(f.read())
        h = int(h_obj.hexdigest(), 16)

    s1 = pow(h, priv_data['dp'], priv_data['p'])
    s2 = pow(h, priv_data['dq'], priv_data['q'])
    h_crt = (priv_data['qinv'] * (s1 - s2)) % priv_data['p']
    s = s2 + h_crt * priv_data['q']

    return base64.b64encode(str(s).encode()).decode()
```

Esta función permite generar un sello digital que garantiza que un archivo no ha sido alterado.

1. **Generación del Hash:** Se aplica el algoritmo SHA2 al contenido del archivo para obtener un "resumen" digital único llamado hash.
2. **Conversión:** El hash se interpreta como un número entero para poder realizar operaciones aritméticas.
3. **Cálculo de Firmas Parciales:** Se utilizan los componentes CRT para trabajar con módulos más pequeños (p y q):
 - a. $s_1 = h^{dp} \bmod p$
 - b. $s_2 = h^{dq} \bmod q$
4. **Recombinación:** Se combinan los resultados parciales para obtener la firma total:
 - a. $h_{crt} = (q_{inv} \times (s_1 - s_2)) \bmod p$
 - b. $s = s_2 + (h_{crt} \times q)$
5. **Salida:** El resultado s es la firma digital, la cual se entrega en formato **Base64**.

Par de llaves generadas:

[illegible]

3. Función de Verificación de Firmas

```
# Funcion de verificacion de firma
def verificar_firma(nombre_archivo, pub_key_b64, s_b64):
    pub_data = json.loads(base64.b64decode(pub_key_b64))

    with open(nombre_archivo, "rb") as f:
        h_prime_obj = SHA256.new(f.read())
        h_prime = int(h_prime_obj.hexdigest(), 16)

    s = int(base64.b64decode(s_b64).decode())
    h_recovered = pow(s, pub_data['e'], pub_data['n'])

    return h_recovered == h_prime
```

Este paso garantiza que el mensaje no fue alterado y que realmente proviene del dueño de la llave privada.

1. **Hash Local:** Se toma el mensaje M recibido y genera su propio hash h' utilizando el mismo SHA2.
2. **Recuperación del Hash Original:** Utilizando la llave pública e y el módulo n, se realiza la operación inversa sobre la firma recibida:

$$h = s^e \bmod n$$

3. **Comparación Final:** Se comparan los valores h (el recuperado de la firma) y h' (el generado localmente).
4. **Veredicto:** Si $h = h'$, la firma se considera válida y el mensaje íntegro; de lo contrario, se rechaza.

Par de llaves generadas:

```
--- LABORATORIO 7: FIRMA RSA CON CRT ---
1. Estudiante A: Generar llaves y firmar mensaje
2. Estudiante B: Verificar firma de un compañero
3. Salir
Seleccione una opción: 2

[Estudiante B] Verificación de firma...
Archivo del mensaje: Gato.txt
Archivo de la llave pública: kpub
Archivo de la firma: firmaGato

[RESULTADO] La firma es VÁLIDA.
```